Internationalization (i18n) — Planning Hub

Status: Planning (pre-implementation) **Initiated:** 2026-04-29 **Target milestone:** French live for ABQ BioPark (June 1–5, 2026); German + Spanish post-ABQ **Drivers:** SHOAL partnership conversation at ABQ benefits from a French-capable platform. German and Spanish broaden the European zoo / aquarium audience that already participates in EAZA EEPs and Citizen Conservation programs.

This folder is the canonical reference for the platform's multilingual rollout. Any new session, any agent, should read this file first.

Why this initiative exists

The platform is currently English-only across every surface: public site, researcher views, coordinator dashboard, Django admin, transactional emails. Two pressures change that now:

1. **ABQ workshop, June 1–5.** A French-capable site materially helps the SHOAL conversation and the broader Francophone-Africa freshwater audience. Madagascar's official languages are Malagasy and French; many in-country partners read French more comfortably than English.
2. **European participation.** EAZA EEP participants (whose entry guide already lives in `docs/`) include a large German-reading population, and Spanish speakers across European and Latin American zoos are a real and growing user segment.

Adding three languages also forces us to do the work properly **once**, with a real translation workflow, rather than accumulating ad-hoc translations of fragments later.

Scope

In scope (all locales — English baseline + French + German + Spanish):

- Every public surface: hero, species directory, species profile, map, About, glossaries, contribute page.
- Every authenticated surface: account pages, coordinator dashboard, researcher views.
- Django admin (custom labels and the side-by-side translation review screens).
- Transactional emails (verify, password reset, coordinator notifications).
- Long-form content: species `description` , `ecology_notes` , `distribution_narrative` , `morphology` . About page narrative. IUCN/CARES glossary entries.
- Locale-aware metadata: `<title>` , `<meta description>` , OpenGraph, `<link rel="alternate" hreflang>` , JSON-LD where present.
- Locale-aware fonts: widen subset to `latin-ext` to cover German umlauts and Spanish/French diacritics consistently.

Out of scope (for this initiative):

- Right-to-left (RTL) languages. None of FR/DE/ES require it; if Arabic is added later it's a separate effort.
- Translation of code, code comments, developer documentation, commit messages, or `docs/planning/` .
- Translation of Latin scientific names. Genus and species names stay Latin everywhere.
- Translation of Malagasy place names in body text. Italicized first appearance, optional gloss; not translated.

- Locale-specific units, number formats, or date formats beyond what `next-intl` and Django's `LANGUAGE_CODE`-driven formatting do for free.

Decisions locked in (2026-04-29)

These are agreed between Alex and Claude before the architecture review. Agents should treat these as constraints, not suggestions.

D1 — Path-prefix routing, English at root

- Default English: `/species/123` (no prefix). Preserves all existing public links, GBIF citations, bookmarks, indexed pages.
- Other locales prefixed: `/fr/species/123`, `/de/species/123`, `/es/species/123`.
- Implemented via `next-intl` middleware with `localePrefix: "as-needed"`.
- Every page emits `<link rel="alternate" hreflang="..." href="...">` for all four locales plus `x-default → /`.
- Rationale: SEO-neutral, link-stable, matches IUCN Red List and Wikipedia conventions. Always-prefix (`/en/...`) was considered and rejected — it would break every existing inbound link.

D2 — Frontend: `next-intl` v3

- App Router-native, server-component support, path-prefix middleware built in.
- Message catalogs as JSON under `frontend/messages/{en,fr,de,es}.json`, namespaced by feature.
- Server components use `getTranslations()`; client islands use `useTranslations()`. Default to server.
- Locale switcher is a small client island in the header.

D3 — Backend: Django `LocaleMiddleware` + `django-modeltranslation`

- Standard `gettext_lazy` for all model `verbose_name`, `help_text`, choice display labels, and form labels.
- `django-modeltranslation` for translatable model fields. It adds suffixed columns (`description_fr`, `description_de`, `description_es`) rather than join tables. Simpler queries, plays well with Django admin out of the box, fine at the scale of ~79 species.
- Translatable fields (initial set): `Species.description`, `Species.ecology_notes`, `Species.distribution_narrative`, `Species.morphology`, `Taxon.common_family_name`, plus glossary entries and any `Page` / static content models that exist or get added.
- `CommonName` already has a per-row `language` field — keep as-is. It is the source of truth for vernacular names; modeltranslation handles long-form prose.

D4 — API: `Accept-Language`-driven, locale resolved server-side

- DRF reads `Accept-Language` (set by the frontend from the URL prefix) and returns localized fields.
- `?lang=fr` query parameter is supported as an override (useful for debugging, curl, and alternate-locale fetches from a single page).
- Serializers return the requested locale's value, falling back to English when the requested locale is missing or below `human_approved` status (see D6).
- No separate `/fr/api/...` URL space. Same API, different `Accept-Language`.

D5 — Machine translation: DeepL API

- DeepL chosen for FR/DE quality. Spanish is comparable to Google.

- Start on DeepL API Free (500K chars/month, free with credit-card verification). Upgrade to API Pro Starter (~€5/mo + ~€20 per million chars) when usage crosses ~400K.
- API key stored as `DEEPL_API_KEY` in `backend/.env`. Endpoint switches between `api-free.deepl.com` and `api.deepl.com` automatically based on the `:fx` suffix on the key.
- Glossary support (Advanced tier, ~€20/mo) deferred. We enforce locked conservation terms in post-processing instead, until MT mistranslations of domain terms become a recurring issue worth paying for.

D6 — Translation pipeline: four-state per field per locale

Each translatable field carries a status per locale:

1. `mt_draft` — DeepL output, not yet reviewed.
2. `writer_reviewed` — `@conservation-writer` agent has reviewed for voice, idiom, and domain terminology.
3. `human_approved` — Alex or a colleague has approved the writer's revision.
4. `published` — same as `human_approved` for now; reserved for future workflow if we add a "scheduled to go live" gate.

Public site rendering rule: only `human_approved` or `published` content is shown in the requested locale. Anything below that falls back to English. This means a partial translation never embarrasses us — French readers see the French content that's been reviewed, and English for anything that hasn't been.

The `fallback to English` rule applies per field, not per page. A species profile in French may render French `description` and English `ecology_notes` on the same page. The page tags fallback content with a discreet "(English)" badge so readers understand they're seeing the source.

D7 — Side-by-side review UI in Django admin

- Custom admin view with EN on the left, target locale on the right, per field.
- Filterable by locale, status, model, and date.
- Approve / send-back buttons advance or revert the status.
- Reviewer (Alex / colleagues) does not need to touch raw `_fr` / `_de` / `_es` fields directly unless they want to; the side-by-side is the primary entry point.
- Free-form `_fr` / `_de` / `_es` field editing remains available in the standard model admin for power-user fixes.

D8 — Conservation-writer agent extended for FR/DE/ES voice review

- Agent definition updated to accept a target locale and the source EN content.
- Agent reviews MT drafts for voice consistency, idiomatic correctness, and conservation-domain terminology — it does not translate from scratch.
- Agent maintains a per-locale glossary of locked terms (IUCN categories, "ex-situ", "studbook", "CARES priority list", etc.) in `docs/planning/i18n/glossaries/`.

D9 — Feature-flag gating throughout

- Single env var `NEXT_PUBLIC_FEATURE_I18N` enables the locale switcher and the `/fr/`, `/de/`, `/es/` route prefixes on the public site.
- Optional per-locale flags (`NEXT_PUBLIC_FEATURE_I18N_FR`, etc.) so French can go live before German is ready.
- Backend `LocaleMiddleware` is on regardless — it's harmless when no locales are advertised — so the admin review workflow can run before public surfaces flip.
- This matches the Gate 11 auth pattern (`NEXT_PUBLIC_FEATURE_AUTH`) and replaces the long-running parallel branch / code-freeze pattern that was originally proposed. Each gate merges to

main behind the flag.

D10 — Gate split

Six gates. L1 establishes the framework. L2/L3 deliver French in time for ABQ. L4 polishes and adds admin/staff surfaces. L5/L6 are German and Spanish post-ABQ. L7 backfills full-corpus species translations.

Each gate is its own short-lived branch off main, merged behind the flag when complete. No gate merges into the next; each merges to main.

Gate	Scope	Branch	Target
L1	Framework: routing, middleware, modeltranslation, switcher, fonts, message catalog extraction (EN baseline) — ✅ **shipped 2026-04-29 (PR #140 + Wave 4)**	`gate/L1-i18n-framework`	May 8
L2	French UI translation (chrome only, all surfaces)	`gate/L2-i18n-french-ui`	May 15
L3	French content (species, About, glossaries) + MT pipeline + admin review UI	`gate/L3-i18n-french-content`	May 22
L4	French polish + admin / coordinator surfaces + emails + flag-flip prep	`gate/L4-i18n-french-staff`	May 29
—	**ABQ window** — French live and demoable	—	Jun 1–5
L5	German UI + content (full corpus through pipeline)	`gate/L5-i18n-german`	post-ABQ
L6	Spanish UI + content (full corpus through pipeline)	`gate/L6-i18n-spanish`	post-ABQ
L7	Backfill any remaining species/content gaps in all locales	`gate/L7-i18n-backfill`	post-ABQ

D11 — All ~79 species translated, batched by family

Gate L3 translates the full species corpus, not a priority subset.

- MT runs once for the whole corpus per language (~100K chars; minutes of API time).
- `@conservation-writer` reviews in family-grouped batches: Bedotiidae, Cichlidae (Paretroplus + Ptychochromis et al.), Aplocheilidae, Anchariidae, gobies, cave fish, others.
- Human approval (Alex + colleagues) follows the same family batching. This is the bottleneck — colleague availability shapes the actual ship date for L3.

Resolved decisions for the architecture review

These supplement D1–D11. They were resolved with Alex on 2026-04-29 after walking through the open questions. The architecture agent **verifies the implementation patterns and produces technical depth**; it does not re-litigate the choices.

D12 — Locale-aware caching follows the auth-tier pattern

- The cache key for any cacheable response must include the active locale; locale-aware fetches must pass `revalidate: 0` (or sit on `force-dynamic` pages) — same defense-in-depth pattern the auth gate uses for tier-aware fetches per `CLAUDE.md`.
- This is invisible plumbing, not user-facing. The user-facing locale switcher is story S7 in Gate L1 (`<LocaleSwitcher />` in the header).
- **Architect's job:** confirm `next-intl` middleware emits the right `Vary` header / cache key on Vercel; document the discipline in the L1 spec where the auth `revalidate: 0` rule is also documented; flag any route that already caches and would need an audit.

D13 — `hreflang` via shared `generateMetadata` helper

- Every page emits `hreflang` for all four locales plus `x-default` (per S8).
- Implementation: a shared helper called from each page's `generateMetadata()`, not duplicated per route.
- Canonical URLs paired with the helper so `<link rel="canonical">` always points at the correct locale-specific URL.
- **Architect's job:** pick the helper's location (e.g., `frontend/lib/seo.ts` or `frontend/lib/metadata.ts`), confirm that the existing metadata in `frontend/app/layout.tsx` and any per-route `generateMetadata` overrides compose cleanly with it.

D14 — DRF locale resolution via Django's `LocaleMiddleware`

- `Accept-Language` header (set by the frontend from the URL prefix) drives `request.LANGUAGE_CODE`; modeltranslation reads from the active language automatically.
- `?lang=fr` query parameter override is supported (S3) but not required for normal use.
- **Architect's job:** confirm interaction with the auth middleware (`TokenAuthentication`, `SessionAuthentication`), document any required `MIDDLEWARE` ordering, verify the existing service-token / session-token paths still work under the new middleware stack.

D15 — Modeltranslation: keep `description` as the implicit English column

- No rename of existing columns. Modeltranslation aliases the unsuffixed column to the default language (`en`) — `Species.description` continues to be the English value, with new `description_fr`, `description_de`, `description_es` siblings.
- Existing English content is preserved on the existing column with no data migration beyond schema additions.
- **Architect's job:** confirm migration shape (one `AddField` per new column per registered model, all `null=True, blank=True`); confirm modeltranslation's admin integration shows the four-locale tabs as expected; flag any DRF serializer that explicitly references `description` to ensure the active-language read continues to work.

D16 — `TranslationStatus` as a separate model

- One row per `(content_type, object_id, field, locale)`, per S4.
- Flexible across models, normalized, queryable for the L3 admin filter UI ("show me everything in `mt_draft` for French species descriptions").
- **Architect's job:** index strategy (composite index on `(content_type, object_id)` plus a covering index for status filters), query patterns for the side-by-side admin review screen in Gate L3, signal/hook strategy for keeping `TranslationStatus` rows in sync when a translatable field is edited directly.

D17 — Email localization: per-User locale + send-time override

- Add a `User.locale` field (CharField, choices = LANGUAGES, default `"en"`) in Gate L4 when emails are first localized.
- Email-sending helpers accept an optional `locale=` kwarg that overrides the user's stored preference (useful for password-reset flows where the user is identified by email before their stored locale is loaded, or for transactional emails sent from a context that already knows the right locale).
- **Architect's job:** design the helper signature (`send_translated_email(user_or_email, template, context, locale=None)` or similar); pick where Django's `translation.override(locale)` wraps the template render; confirm the existing email sender is compatible.

D18 — Sitemap with cross-locale `xhtml:link` annotations

- Single sitemap with each URL's locale variants annotated via `<xhtml:link rel="alternate" hreflang="...">`.
- Modern Google-recommended pattern; expresses the relationship between locale variants better than per-locale sitemaps.
- **Architect's job:** identify the existing sitemap generator (Django's `django.contrib.sitemaps`, a Next.js-side route, or absent); if absent, propose where the sitemap lives; confirm the `xhtml:link` annotations are emitted correctly per URL.

Architecture agent next steps

The architecture agent is invoked after this hub merges to `main`. It produces `docs/planning/architecture/i18n-architecture.md` on a `docs/i18n-architecture` branch covering:

- The eight resolved decisions above, with the technical detail each "Architect's job" line calls out.
- Any new questions / risks that surface from grounding the plan in the current codebase (`frontend/middleware.ts`, `backend/config/settings/base.py`, the existing serializers, etc.).
- A concrete implementation order for Gate L1's twelve stories — which story unblocks which, where the parallelizable work is.
- Any required updates to `gate-L1-framework.md` based on what the architect surfaces.

Architecture / non-goals worth naming

- **No CMS.** Translatable content lives in Django models and is edited in Django admin. We are not adding Contentful, Sanity, Strapi, or similar. The side-by-side review screen is custom Django admin, not a third-party tool.
- **No translation-management SaaS.** We are not paying for Crowdin, Lokalise, or Transifex. The MT-then-review pipeline runs in-house.
- **No machine learning beyond DeepL.** No fine-tuning, no custom models, no LLM-as-translator (the conservation-writer agent reviews MT output, it doesn't replace MT).
- **No locale-specific feature toggles.** A feature is either on or off; it is not "on in French only." If a feature legitimately depends on locale (e.g., a French-only press kit), that's a separate page, not a flagged feature.

L3+ future-work parking lot (architect-surfaced)

Carried forward from `docs/planning/architecture/i18n-architecture.md` §12–13. Each item is real work that does not block L1; agents picking up L3+ should consult the architecture doc for context.

- **Localized search across translatable fields (Q-NEW2, L3+).** `SpeciesViewSet.search_fields` references locale-agnostic columns today. When French content lands in L3, a French-speaking user searching `"rouge"` should match against `description_fr`. This is a UX call (rank stability, fallback behavior, multilingual term overlap) handled in L3 or later.
- **Glossary directory for the conservation-writer agent (L3 prereq).** `.claude/agents/conservation-writer.md` references `docs/planning/i18n/glossaries/`. L3 must create that directory and seed it with empty per-locale glossary files before the writer agent is invoked against MT drafts. Otherwise the agent fails on missing inputs.
- **DeepL availability fallback (L3).** Pipeline must handle DeepL request failures gracefully: mark the `TranslationStatus` row as `mt_draft` with `notes` populated, retry job picks it up next pass. Architecture doc §13.6.
- **Account-page locale field (L4).** When `User.locale` is added in L4, the account-page profile editor must surface it so users can self-serve. The L4 spec describes the email work but not the corresponding UI. Architecture doc §13.7.
- **`Taxon` admin TranslationAdmin inheritance (cosmetic, L1 implementer note).** S2 registers `Taxon.common_family_name` as translatable. The current `Taxon` admin class needs to inherit `TranslationAdmin` (or `TabbedTranslationAdmin`) for the locale tabs to render correctly. Architecture doc §12 Q-NEW1.
- **`force-dynamic` × locale render cost (L4 monitor).** Every protected route runs `next-intl` middleware *plus* the auth gate; `force-dynamic` pages bypass the locale-aware ISR cache entirely. Acceptable in L1; if perf regresses post-L1, the fix is to push heavy data fetches into route handlers that *do* cache per locale. Architecture doc §13.1–13.2.
- **`*_locale_actual` keys are permanent (architectural commitment).** Once shipped, removing them is a breaking API change. Architecture doc §13.4.

How to pick this up in a new session

1. Read this file.
2. Read the gate spec you're working in (`gate-L1-framework.md`, `gate-L2-french-ui.md`, etc.).
3. Read `docs/planning/architecture/i18n-architecture.md` — it contains the technical depth and resolved D12–D18 patterns.
4. Check `git branch` — i18n work happens on `gate/L<n>-i18n-*` branches, never on `main`.
5. If the decisions above (D1–D18) seem wrong for what you're being asked to do, stop and ask — don't override them silently.